

Retina OCT Autoencoder Projesi Anlatım Rehberi

Bu dosya, projeyi hiç bilmeyen birinin bile mantığını anlayıp sözlüde rahatça anlatabilmesi için hazırlandı. Buradaki amaç sadece "ne yaptık" demek değil; aynı zamanda "neden bunu seçtik", "nasıl eğittik", "sonuç ne anlama geliyor" ve "hoca teknik soru sorarsa ne cevap verilir" kısımlarını tek yerde toplamaktır.

1. Projeyi Tek Cümlede Nasıl Anlatırım?

Bu projede, retina OCT görüntülerindeki patolojik örnekleri doğrudan sınıflandırmak yerine, sadece normal görüntüler üzerinde eğitilen bir convolutional autoencoder ile normal anatomiye öğrendik; daha sonra patolojik örneklerin reconstruction error değerlerinin yükselmesini kullanarak anomali tespiti yaptık.

Kısa sözlü versiyon:

"Biz bu projede klasik supervised hastalık sınıflandırması yapmak yerine anomaly detection yaklaşımını seçtik. Modeli sadece normal retina OCT görüntüleriyle eğittik. Böylece model normal anatomiye öğrendi. Testte patolojik görüntüler geldiğinde bunları normal kadar iyi reconstruct edemediği için reconstruction error yükseldi. Biz de bu hatayı kullanarak patolojik örnekleri tespit ettik."

2. Bu Projenin Hikayesi Ne?

Bu proje aslında medikal görüntü analizindeki iki temel probleme cevap veriyor.

Birinci problem, medikal verilerin etiketlenmesinin zor olmasıdır. Retina OCT gibi görüntüleri doğru yorumlamak uzmanlık gerektirir. Her zaman çok büyük, dengeli ve temiz etiketlenmiş patoloji verisi bulunmaz.

İkinci problem ise klasik denetimli sınıflandırmanın sınırlı kalabilmesidir. Eğer model yalnızca belirli hastalık etiketleriyle eğitilirse, eğitimde hiç görmediği farklı bir anormalliği kaçırabilir. Oysa klinik hayatta her sapma önceden etiketlenmiş olmayabilir.

Biz bu nedenle problemi şöyle kurduk:

- Normal anatomiği öğrenelim.
- Normalden sapmayı anomali olarak yakalayalım.
- Modelin mantığı "bu hangi hastalık?" değil, "bu normal mi değil mi?" olsun.

Bu nedenle proje, doğrudan çok sınıflı sınıflandırma değil, `normal-only anomaly detection` mantığıyla tasarlandı.

3. Tam Olarak Hangi Problemi Çözdük?

Problemimiz şuydu:

Retina OCT görüntülerinde normal ile patolojik örnekleri ayırmak.

Ama bunu klasik sınıflandırma gibi kurmadık. Yani amacımız:

- `CNV`
- `DME`
- `DRUSEN`

sınıflarını ayrı ayrı tahmin etmek değildi.

Bunun yerine şu yaklaşımı kullandık:

- Eğitim aşamasında sadece `NORMAL` görüntüler kullanalım.
- Model normal görüntüleri reconstruct etmeyi öğrensin.
- Testte hem `NORMAL` hem patolojik görüntüler verelim.
- Reconstruction error yükselirse bunu anomali sinyali kabul edelim.

Bu yüzden projenin ana görevi, teknik olarak `normal vs patolojik` ayrımı yapan bir anomaly detection sistemidir.

4. Neden Retina OCT Verisi Seçtik?

Retina OCT, göz tabakalarını yüksek çözünürlükte gösteren bir medikal görüntüleme yöntemidir. Retina hastalıklarının erken tespiti klinik açıdan önemlidir; çünkü geç tanı kalıcı görme kaybına yol açabilir.

Bu veri setini seçmemizin nedenleri şunlardır:

- Dersin medikal uygulamalar temasına doğrudan uyuyor.
- Literatürde bilinen ve sık kullanılan bir veri seti.
- Sınıfları net: `NORMAL`, `CNV`, `DME`, `DRUSEN`.
- Hem ara rapor hem de final aşaması için savunulabilir bir deney kurmaya elverişli.

5. Kullandığımız Veri Seti Neydi?

Resmi kaynak olarak Kermany OCT2017 veri kümesini kullandık.

Proje içinde veri şu yapıda yer aldı:

```
data/oct2017/  
  train/  
    NORMAL/  
  test/  
    CNV/  
    DME/  
    DRUSEN/  
    NORMAL/
```

Burada en kritik nokta şudur:

- Eğitimde yalnızca `train/NORMAL` kullanıldı.
- `train/NORMAL` içinden hasta düzeyinde `train` ve `validation` ayrımı yapıldı.
- Testte ise `test/NORMAL`, `test/CNV`, `test/DME`, `test/DRUSEN` birlikte değerlendirildi.

Yani model patolojik sınıfları eğitimde hiç görmedi; onları sadece testte gördü.

6. Veri Sayıları Nasıldı?

Gerçek koşuda veri dağılımı şu şekildeydi:

- Eğitim: `40,715` normal görüntü, `2,747` hasta
- Doğrulama: `10,425` normal görüntü, `687` hasta
- Test NORMAL: `250` görüntü, `171` hasta
- Test CNV: `250` görüntü, `178` hasta
- Test DME: `250` görüntü, `167` hasta
- Test DRUSEN: `250` görüntü, `169` hasta

Bu sayılar bize şunu sağladı:

- Model çok sayıda normal örnek gördü.
- Test seti sınıflar arasında dengeli kaldı.
- Sonuçları sınıf bazında yorumlamak mümkün oldu.

7. Neden Hasta Düzeyinde Split Yaptık?

Bu, projenin en önemli metodolojik kararlarından biridir.

Eğer train ve validation ayrımını görüntü düzeyinde yapsaydık, aynı hastaya ait çok benzer kesitler hem eğitimde hem doğrulamada yer alabilirdi. Bu da modelin gerçekten genelleme yapmadan, aynı hastanın benzer örneklerini tekrar görmesi anlamına gelirdi. Böyle bir durumda validation sonucu gerçekte olduğundan daha iyi çıkabilirdi. Bu risk `data leakage` olarak bilinir.

Biz bunu engellemek için hasta kimliğini dosya adından çıkardık. Veri setindeki dosya adları şu mantığa benzer:

`CNV-3156-13.jpeg`

Buradaki orta bölüm hasta kimliğidir. Kod tarafında regex ile bu kimliği parse ettik ve `train/NORMAL` içindeki ayrımı bu hasta kimliği üzerinden yaptık. Böylece aynı hastaya ait görüntüler train ve validation tarafına karışmadı.

Hoca "neden patient-level validation yaptınız?" derse verilecek güzel cevap:

"Çünkü image-level rastgele split, aynı hastaya ait benzer kesitlerin hem train hem validation tarafında yer almasına neden olabilir. Bu da validation performansını yapay olarak yükseltir. Biz daha gerçekçi bir deney için hasta düzeyinde split kullandık."

8. Ön İşleme Aşamasında Ne Yaptık?

Modelden önce görüntülere şu işlemler uygulandı:

- Görüntüler gri tonlamaya çevrildi
- Tüm görüntüler `128x128` boyutuna yeniden ölçeklendirildi
- Piksel değerleri `[0,1]` aralığına normalize edildi

Buradaki mantık şuydu:

- Autoencoder sabit boyutlu giriş ister
- OCT görüntüsü için gri tonlama yeterli bilgiyi taşır
- `128x128`, hesaplama maliyeti ile görsel bilgi arasında makul bir dengedir

Hoca "neden daha büyük çözünürlük kullanmadınız?" derse:

"Bu aşamada amacımız en ağır modeli kurmak değil, çalışan ve savunulabilir bir baseline elde etmektir. 128x128 çözünürlük, eğitim süresini makul tutarken temel yapısal bilgiyi koruyan pratik bir başlangıç seçimi oldu."

9. Model Olarak Tam Olarak Neyi Kullandık?

Modelimiz bir `convolutional autoencoder`.

Autoencoder iki temel parçadan oluşur:

- `encoder`: görüntüyü daha küçük bir latent temsile sıkıştırır
- `decoder`: bu latent temsilden görüntüyü yeniden oluşturmaya çalışır

Biz neden bunu seçtik?

Çünkü autoencoder normal örneklerde düşük reconstruction error üretir. Testte gelen bir görüntü normal anatomiden belirgin şekilde sapıyorsa model bunu normal kadar iyi reconstruct edemez ve hata yükselir. Bu da anomaly detection için kullanılabilecek doğal bir sinyal oluşturur.

10. Model Mimarisi Nasıldı?

Model 4 blokluk bir convolutional yapıya sahipti.

Encoder tarafı:

- `Conv2d(1 -> 32) + ReLU + MaxPool`
- `Conv2d(32 -> 64) + ReLU + MaxPool`
- `Conv2d(64 -> 128) + ReLU + MaxPool`
- `Conv2d(128 -> 256) + ReLU + MaxPool`

Giriş boyutu `128x128` olduğundan, dört kez pooling sonrasında uzaysal boyut `8x8` seviyesine düşüyor. Bu noktada özellik haritası:

- `256 x 8 x 8`

oluyor.

Ardından tam bağlantılı katmanla latent uzaya geçiliyor:

- `Linear(256\*8\*8 -> 128)`

Buradaki `128`, modelin `latent\_dim` değeridir.

Decoder tarafı ise şu şekilde çalışıyor:

- `Linear(128 -> 256\*8\*8)`
- `ConvTranspose2d(256 -> 128)`
- `ConvTranspose2d(128 -> 64)`
- `ConvTranspose2d(64 -> 32)`
- `ConvTranspose2d(32 -> 16)`
- `Conv2d(16 -> 1) + Sigmoid`

Özet mantık:

`giriş görüntüsü -> sıkıştırma -> latent temsil -> yeniden oluşturma`

Modelin toplam parametre sayısı yaklaşık `4.77 milyon`dur.

11. Tam Olarak Nasıl Eğittik?

Bu kısım sözlüde çok önemli olacaktır. Çünkü hoca genelde burada şu tür sorular sorar:

- Hangi loss kullandınız?
- Hangi optimizer kullandınız?
- Kaç epoch eğittiniz?
- Validation nasıl yaptınız?

Bu nedenle aşağıdaki bölümü net biçimde bilmek gerekir.

Eğitim Ayarları

- Görüntü boyutu: `128x128`

- Latent boyut: `128`
- Batch size: `32`
- Optimizer: `Adam`
- Learning rate: `0.001`
- Loss function: `MSE reconstruction loss`
- Maksimum epoch: `40`
- Early stopping patience: `8`
- Validation oranı: `0.20`
- Random seed: `42`
- Num workers: `0`

Loss Nasıl Hesaplandı?

Model çıktısı ile orijinal görüntü arasındaki ortalama karesel hata kullanıldı:

$$\text{MSE} = \text{mean}((x - x_{\text{hat}})^2)$$

Burada:

- `x` orijinal görüntü
- `x\_hat` modelin reconstruct ettiği görüntü

Amaç, normal görüntülerde bu reconstruction loss değerini mümkün olduğunca düşürmekti.

Eğitim Akışı Nasıl İşledi?

1. Model sadece `train/NORMAL` görüntüleriyle eğitildi.
2. Her epoch sonunda `validation` normal görüntülerinde loss hesaplandı.
3. Validation loss en iyi olduğunda model checkpoint olarak kaydedildi.
4. Eğer validation loss `8` epoch boyunca iyileşmezse `early stopping` devreye girdi.
5. Eğitim sonunda en iyi checkpoint tekrar yüklenerek test aşamasına geçildi.

12. Gerçek Eğitim Koşusu Nasıl Sonuçlandı?

Gerçek deneyde:

- En iyi epoch: `37`
- En iyi validation loss: `0.000745`
- Toplam eğitim süresi: `6581.56` saniye, yani yaklaşık `109.7` dakika

GPU tarafında eğitim, yerel PyTorch ortamında `NVIDIA GeForce RTX 4060 Laptop GPU` üzerinde çalıştırıldı.

Hoca "neden uzun sürdü?" derse:

"Model GPU üzerinde çalıştı ama veri yükleme tarafında `num\_workers=0` olduğu için CPU darboğazı oluştu. Bu nedenle GPU sürekli tam kapasite dolmadı. Buna rağmen gerçek deney başarıyla tamamlandı."

13. Neden MSE Kullandık?

MSE seçmemizin temel nedeni, reconstruction tabanlı modeller için en sade ve en savunulabilir başlangıç seçeneklerinden biri olmasıdır.

Avantajları:

- Uygulaması basit
- Yorumlaması kolay
- Reconstruction kalitesini doğrudan optimize ediyor
- Baseline sistem için uygun

Hoca "neden SSIM ya da başka loss denemediniz?" derse:

"Bu aşamada önce sade, çalışır ve savunulabilir bir baseline kurmak istedik. MSE reconstruction loss bu amaç için uygun bir başlangıç sundu. Daha gelişmiş loss fonksiyonları final geliştirmesi olarak eklenebilir."

14. Karar Mekanizması Nasıl Çalıştı?

Model, test aşamasında doğrudan "normal" veya "patolojik" etiketi üretmiyor. Önce her görüntü için sayısal bir reconstruction error değeri üretiyor. Sonra bu hata değeri bir eşikle karşılaştırılıyor.

Karar kuralı şu:

- Reconstruction error eşikten büyükse: `anomaly`
- Reconstruction error eşikten küçük veya eşitse: `normal`

Burada en kritik nokta, eşik değerinin test setine bakılarak seçilmemesidir.

Eşikler sadece `validation/NORMAL` görüntülerindeki reconstruction error dağılımından üretildi. Böylece test seti yalnızca son değerlendirme için kullanıldı.

Denenen persentiller:

- `p95`
- `p97`
- `p99`

15. Neden Son Olarak p95 Seçtik?

Çünkü `p95`, `p97` ve `p99` ile karşılaştırıldığında daha dengeli bir çalışma noktası sundu.

Karşılaştırma mantığı:

- `p95`: recall daha yüksek, F1 daha iyi
- `p97`: false positive biraz azalıyor ama recall düşüyor
- `p99`: çok muhafazakar davranıyor, precision yüksek ama çok fazla patolojik örnek kaçıyor

Bu yüzden ara rapordaki ana işletim noktası olarak `p95` seçildi.

Hoca "neden p97 değil?" derse:

"p97 daha düşük false positive verse de recall kaybı belirginleşti. Biz bu aşamada precision ve recall arasında daha dengeli bir sonuç verdiği için p95'i tercih ettik."

16. Hangi Metrikleri Hesapladık?

Tek bir metrikle yetinmedik. Çünkü anomaly detection probleminde sadece accuracy'ye bakmak yanıltıcı olabilir. Bu nedenle birden fazla metrik raporladık:

- `AUROC`
- `Accuracy`
- `Precision`
- `Recall`
- `F1-score`
- `FPR`
- `Confusion matrix`

Bu metriklerin anlamı:

- `AUROC`: modelin normal ve patolojik örnekleri genel olarak ne kadar iyi ayırabildiğini gösterir
- `Accuracy`: toplam doğru tahmin oranı
- `Precision`: model patolojik dediğinde ne kadar doğru dediği
- `Recall`: gerçek patolojik örneklerin ne kadarını yakalayabildiği
- `F1-score`: precision ve recall dengesini gösterir
- `FPR`: normal görüntüleri yanlışlıkla patolojik deme oranı

17. Gerçek Sonuçlar Neydi?

Seçilen `p95` eşiği için sonuçlarımız:

- `AUROC = 0.9108`
- `Accuracy = 0.7670`
- `Precision = 0.9743`
- `Recall = 0.7080`
- `F1-score = 0.8201`
- `FPR = 0.0560`

Confusion matrix:

- `TN = 236`
- `FP = 14`
- `FN = 219`
- `TP = 531`

Bu sayılar ne demek?

- `250` normal test görüntüsünün `236` tanesini doğru şekilde normal bulduk
- Sadece `14` normal görüntüyü yanlışlıkla patolojik dedik
- `750` patolojik örneğin `531` tanesini yakaladık
- `219` patolojik örneği kaçırdık

Doğru yorum şu olur:

"Model patolojik dediğinde çoğu zaman doğru söylüyor; fakat tüm patolojik örnekleri eksiksiz yakalayamıyor."

18. Sınıf Bazında Neler Gördük?

Sınıf bazlı ortalama reconstruction error değerleri:

- `CNV`: `0.002921`
- `DME`: `0.002592`
- `DRUSEN`: `0.001317`
- `NORMAL`: `0.000864`

Bu tablo bize şunu gösteriyor:

- `CNV` ve `DME` normalden daha belirgin sapma gösterdiği için reconstruction error yüksek
- `DRUSEN`, diğer patolojilere göre normale daha yakın kaldı
- `NORMAL` sınıfı en düşük reconstruction error'a sahip

Yani modelin en zorlandığı patoloji `DRUSEN` oldu.

Tespit Sayıları

`p95` eşik noktasında:

- `CNV`: `244 / 250`
- `DME`: `204 / 250`
- `DRUSEN`: `83 / 250`

Bu sonuç sözlüde şöyle yorumlanabilir:

"CNV ve DME sınıfları yapısal olarak normal retinadan daha belirgin sapmalar içerdiği için model bunları daha rahat anomali olarak işaretledi. DRUSEN ise hata dağılımı bakımından normal sınıfa daha yakın kaldığı için daha zor tespit edildi."

19. Projede Hangi Dosyalar Ne İşe Yarıyor?

Projede uçtan uca bir pipeline kurduk. Kod tarafı şu şekilde ayrıldı:

- `main.py`: tüm akışı tek komutla çalıştırıyor
- `data\_utils.py`: veri hazırlama, transform ve patient-level split
- `model.py`: convolutional autoencoder mimarisi
- `train.py`: eğitim döngüsü ve early stopping
- `evaluate.py`: threshold, metrik, grafik ve değerlendirme
- `report\_builder.py`: rapor için içerik ve çıktı üretimi

Bu önemli çünkü proje sadece bir fikir veya tek bir notebook değil; modüler ve tekrar çalıştırılabilir bir sistem olarak kuruldu.

20. Tam Olarak Hangi Çıktıları Ürettik?

Bu proje sonucunda sadece metrik değil, birden fazla somut çıktı üretildi:

- Eğitim kaybı grafiği
- ROC eğrisi
- Reconstruction örnekleri
- Error histogramı
- Confusion matrix
- Threshold comparison tablosu
- Sınıf bazlı reconstruction summary
- Ara rapor dosyaları

Yani elimizde hem çalışan kod hem de bunu destekleyen deney çıktıları var.

21. Projenin Güçlü Tarafları Neler?

- Problem tanımı net: `normal vs patolojik`
- Ders içeriğiyle uyumlu bir yöntem kullanıldı: `autoencoder`
- Veri seti literatürde bilinen bir kaynak
- `patient-level validation` ile metodolojik dikkat gösterildi
- Eşik seçimi testten değil validation dağılımından yapıldı
- Sonuçlar gerçek veri üzerinde elde edildi
- Reconstruction error mantığı açıklanabilir olduğu için modelin davranışı yorumlanabiliyor

22. Projenin Sınırları Neler?

Hoca "eksik yönü ne?" diye sorarsa şu noktalar söylenebilir:

- Model bir baseline düzeyinde; daha gelişmiş mimariler henüz denenmedi
- `128x128` çözünürlük daha ince ayrıntıları kaybettiriyor olabilir
- `MSE` her zaman en iyi anomaly sinyali olmayabilir
- `DRUSEN` sınıfında performans görece düşük
- `num\_workers=0` olduğu için eğitim verimliliği sınırlı kaldı
- Bu sistem doğrudan klinik kullanıma hazır bir ürün değil; araştırma odaklı bir baseline

Bu kısmın güzel bir sözlü özeti şu olabilir:

"Bizim projemiz çalışan bir baseline sistem sunuyor. Klinik kullanıma hazır kusursuz bir sistem iddia etmiyoruz. Ama normal-only anomaly detection mantığının retina OCT verisinde gerçek deneyle çalıştığını göstermiş olduk."

23. Bu Proje Nasıl Geliştirilebilir?

Her ne kadar şu an ara rapor odağında olsak da, projenin gelişim yönü açık:

- `Variational Autoencoder (VAE)` eklenebilir
- `SSIM` veya karma loss fonksiyonları denenebilir
- Daha yüksek çözünürlük kullanılabilir
- `latent\_dim`, `batch size`, `image\_size` için ablasyon çalışması yapılabilir
- `kalıntı (residual) haritaları` ile anomaly localization denenebilir
- Autoencoder ile başka anomaly detection modelleri karşılaştırılabilir

24. Yarın Sözlüde Takip Edebileceğim Uzun Anlatım Akışı

Aşağıdaki sırayla anlatırsan proje doğal ve kontrollü görünür.

Giriş

"Biz medikal uygulamalar teması altında retina OCT görüntülerinde anomaly detection problemi seçtik. Amacımız, patolojik sınıfları doğrudan supervised şekilde ayırmak değil; yalnızca normal görüntülerle eğitilen bir modelin patolojik sapmaları reconstruction error ile yakalayıp yakalayamayacağını göstermekteydi."

Motivasyon

"Bu yaklaşımı seçmemizin nedeni, medikal verilerde etiketli patoloji verisinin her zaman yeterli olmaması ve modelin eğitimde görmediği yeni anomalilerle de karşılaşabilmesidir. Bu yüzden normal anatomiyi öğrenmek ve normallikten sapmayı anomali saymak daha esnek bir yaklaşım sundu."

Veri Seti

"Kermany OCT2017 veri setini kullandık. Eğitimde sadece NORMAL görüntüleri kullandık. Bu görüntüleri de hasta düzeyinde train ve validation olarak ayırdık. Test aşamasında NORMAL, CNV, DME ve DRUSEN görüntülerini birlikte değerlendirdik."

Patient-level split

"Burada özellikle patient-level validation kullandık. Çünkü image-level rastgele split, aynı hastaya ait benzer görüntülerin hem eğitim hem doğrulama tarafında görünmesine neden olabilir ve data leakage yaratabilir."

Model

"Model olarak 4 bloklı convolutional autoencoder kullandık. Encoder görüntüyü sıkıştırdı, decoder bu temsilden yeniden oluşturdu. Latent boyutu 128 olarak seçildi. Model normal görüntülerde düşük reconstruction error üretmeyi öğrendi; patolojik görüntülerde ise hata yükseldi."

Eğitim

"Modeli 128x128 gri tonlamalı görüntülerle, batch size 32, Adam optimizer, learning rate 0.001 ve MSE reconstruction loss ile eğittik. Maksimum 40 epoch çalıştırdık, early stopping patience değerini 8 olarak kullandık."

Karar mekanizması

"Her test görüntüsü için reconstruction error hesaplandı. Sonra validation normal dağılımından p95, p97 ve p99 eşikleri çıkardık. En dengeli sonuç p95'te geldiği için ana işletim noktası olarak onu seçtik."

Sonuçlar

"Gerçek deneyde p95 eşik noktasında AUROC 0.9108 ve F1-score 0.8201 elde ettik. Precision 0.9743 olduğu için model patolojik dediğinde çoğu zaman doğruydı. Ancak recall 0.708 seviyesinde kaldığı için tüm patolojik örnekleri eksiksiz yakalayamadı. Sınıf bazında CNV ve DME daha rahat tespit edilirken DRUSEN daha zor çıktı."

Yorum

"Bu nedenle yaklaşımın anlamlı ve umut verici bir baseline sunduğunu söyleyebiliriz. Özellikle normal-only anomaly detection mantığının retina OCT verisi üzerinde gerçek deneyle çalıştığını gösterdik."

25. Hoca Sorarsa Hazır Cevaplar

Neden supervised classification değil de anomaly detection seçtiniz?

Çünkü amacımız belirli etiketlere bağlı bir sınıflandırma kurmak değil, normal anatomiye öğrenip ondan sapmaları yakalamaktı. Bu yaklaşım etiket eksikliği ve yeni anormallik ihtimali karşısında daha esnek.

Neden sadece NORMAL ile eğittiniz?

Çünkü modelin normal dağılımı öğrenmesini istedik. Patolojik sınıfları da eğitime katsaydık anomaly detection mantığı zayıflardı.

Neden convolutional autoencoder kullandınız?

Çünkü görüntü verisinde uzaysal yapı önemlidir. Convolution katmanları bu yapıyı korur. Autoencoder ise reconstruction tabanlı anomaly detection için doğal bir başlangıç modelidir.

Neden latent boyut 128?

Çok küçük olup bilgiyi aşırı kaybettirmeyen, çok büyük olup modeli gereğinden fazla serbest bırakmayan makul bir ara değer olduğu için.

Neden 128x128 kullandınız?

Hesaplama maliyeti ile yapısal bilgi arasında dengeli bir çözüm olduğu için.

Neden MSE reconstruction loss kullandınız?

Sade, standart ve yorumlanabilir bir baseline elde etmek istediğimiz için.

Neden Adam optimizer kullandınız?

Pratik, kararlı ve görüntü tabanlı derin öğrenme problemlerinde sık kullanılan bir optimizasyon yöntemi olduğu için.

Neden early stopping kullandınız?

Validation loss iyileşmeyi durdurduğunda gereksiz epoch çalıştırmamak ve overfitting riskini azaltmak için.

Eşiği neden validation setten seçtiniz?

Çünkü test setine bakarak eşik seçmek metodolojik olarak yanlıştır. Test yalnızca son değerlendirme için kullanılmalıdır.

Neden p95 seçtiniz?

Çünkü p95, precision ve recall dengesi açısından p97 ve p99'dan daha iyi sonuç verdi.

En zor sınıf hangisiydi?

`DRUSEN`. Çünkü reconstruction error dağılımı normal sınıfa daha yakın kaldı.

Sonuçlar iyi mi?

Bir baseline anomaly detection sistemi için umut verici. AUROC yaklaşık 0.91 ve precision çok yüksek. Ama recall daha da artırılabilir.

Bu proje bitti mi?

Temel haliyle çalışan bir baseline proje tamamlandı. Ama daha güçlü bir final sürümü için yeni modeller ve ek deneyler yapılabilir.

26. Ezberlenmesi Gereken Kısa Sayılar

Yarın öncesi aklında kalması gereken temel rakamlar:

- Veri seti: `Kermany OCT2017`
- Eğitim: sadece `NORMAL`
- Train: `40,715`
- Validation: `10,425`
- Test: her sınıf `250`
- Image size: `128x128`
- Latent dim: `128`
- Batch size: `32`
- Learning rate: `0.001`
- Epoch: `40`
- Patience: `8`
- Loss: `MSE reconstruction loss`
- Optimizer: `Adam`
- Seçilen eşik: `p95`
- AUROC: `0.9108`
- Precision: `0.9743`
- Recall: `0.7080`
- F1-score: `0.8201`
- En zor sınıf: `DRUSEN`
- En iyi epoch: `37`

27. Tek Paragrafta Son Özet

Eğer biri sana "bu projede tam olarak ne yaptınız?" derse şu cevap yeterince güçlü olur:

"Bu projede retina OCT görüntülerinde patolojik örnekleri tespit etmek için yalnızca normal görüntülerle eğitilen bir convolutional autoencoder geliştirdik. Model normal anatomiye reconstruct etmeyi öğrendi; patolojik görüntüler geldiğinde reconstruction error yükseldiği için bunu anomaly sinyali olarak kullandık. Patient-level validation, validation tabanlı threshold seçimi ve gerçek test sonuçlarıyla bu yaklaşımın retina OCT anomaly detection için anlamlı bir baseline sunduğunu gösterdik."